

GOLDEN CHEATSHEET

For Java Streams API - Part 1 to 16 Revision

Concept • Input • Code • Output • Interview Point

Java 8 - Java 21 | Fast visual revision

Revision strategy: Scan the colored title, read the one-line explanation, then memorize the shortest code pattern. Red cards are warnings; purple cards are internals; green cards are preferred practices.

1. FUNDAMENTALS & CREATION

Stream pipeline

Explanation: A Stream connects source, lazy transformations, and one terminal operation.

Input: [1,2,3,4]

```
List<Integer> out = List.of(1,2,3,4).stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * 10)
    .toList();
```

Output: [20, 40]

Interview: Source -> intermediate -> terminal.

Lazy evaluation

Explanation: Intermediate operations execute only after a terminal operation.

Input: [5,15,25]

```
Stream<Integer> s = List.of(5,15,25).stream()
    .filter(n -> { System.out.println(n); return n > 10; });
```

Output: No output

Interview: Add count(), toList(), or another terminal operation to execute.

stream()

Explanation: Create a sequential Stream from a Collection.

Input: [Java, Spring]

```
List.of("Java", "Spring").stream()
    .forEach(System.out::println);
```

Output: Java Spring

Interview: Sequential is the safe default.

parallelStream()

Explanation: Create a parallel Stream using the common ForkJoinPool.

Input: [1,2,3,4]

```
List<Integer> out = List.of(1,2,3,4).parallelStream()
    .map(n -> n * 2).toList();
```

Output: [2, 4, 6, 8]

Interview: Benchmark first; use for large CPU-bound stateless work.

Arrays.stream()

Explanation: Create a Stream from an object or primitive array.

Input: [1,2,3]

```
int sum = Arrays.stream(new int[]{1,2,3}).sum();
```

Output: 6

Interview: Primitive arrays create primitive Streams.

Stream.of() / empty()

Explanation: Create from values or represent no values safely.

Input: A,B / no values

```
long a = Stream.of("A", "B").count();
long b = Stream.empty().count();
```

Output: 2 / 0

Interview: Prefer empty Stream over null.

generate()	iterate()
Explanation: Create independent infinite values using Supplier.	Explanation: Create a sequence where each value depends on the previous one.
Input: Java x 3	Input: start=1, n+1
<pre>List<String> out = Stream.generate(() -> "Java") .limit(3).toList();</pre>	<pre>List<Integer> out = Stream.iterate(1,n -> n+1) .limit(5).toList();</pre>
Output: [Java, Java, Java]	Output: [1, 2, 3, 4, 5]
Interview: Always bound infinite Streams.	Interview: Java 9 overload accepts a predicate.

ofNullable()	Files.lines()
Explanation: Return a 0-or-1 element Stream.	Explanation: Read file lines lazily as a Stream.
Input: null	Input: ERROR OK ERROR
<pre>long count = Stream.ofNullable(null).count();</pre>	<pre>try (Stream<String> lines = Files.lines(path)) { long count = lines.filter(s -> s.contains("ERROR")).count(); }</pre>
Output: 0	Output: 2
Interview: Java 9 null-safe Stream creation.	Interview: Always close I/O Streams with try-with-resources.

2. INTERMEDIATE OPERATIONS

filter()

Explanation: Keep elements that match a Predicate.

Input: [1,2,3,4]

```
List<Integer> out = List.of(1,2,3,4).stream()
    .filter(n -> n % 2 == 0).toList();
```

Output: [2, 4]

Interview: Stateless and lazy; similar to SQL WHERE.

map()

Explanation: Transform every element one-to-one.

Input: [java,spring]

```
List<String> out = List.of("java", "spring").stream()
    .map(String::toUpperCase).toList();
```

Output: [JAVA, SPRING]

Interview: map changes; it does not flatten.

flatMap()

Explanation: Flatten nested Streams into one Stream.

Input: [[1,2],[3,4]]

```
List<Integer> out = List.of(List.of(1,2),List.of(3,4))
    .stream().flatMap(List::stream).toList();
```

Output: [1, 2, 3, 4]

Interview: Use for nested collections and one-to-many relationships.

distinct()

Explanation: Remove duplicates using equals/hashCode.

Input: [1,2,2,3]

```
List<Integer> out = List.of(1,2,2,3).stream()
    .distinct().toList();
```

Output: [1, 2, 3]

Interview: Stateful; custom objects need correct equality.

sorted()

Explanation: Sort by natural order.

Input: [30,10,20]

```
List<Integer> out = List.of(30,10,20).stream()
    .sorted().toList();
```

Output: [10, 20, 30]

Interview: Stateful and may buffer all elements.

sorted(Comparator)

Explanation: Sort by custom order or properties.

Input: [30,10,20]

```
List<Integer> out = List.of(30,10,20).stream()
    .sorted(Comparator.reverseOrder()).toList();
```

Output: [30, 20, 10]

Interview: Use thenComparing for stable multi-field business sorting.

peek()	limit() / skip()
Explanation: Observe elements without transforming them. Input: [5,15] <pre>List<Integer> out = List.of(5,15).stream() .peek(System.out::println).filter(n -> n > 10).toList();</pre> Output: console: 5,15; result: [15] Interview: Debug only; do not put required business side effects here.	Explanation: Take first N / ignore first N. Input: [1,2,3,4,5] <pre>List<Integer> page = List.of(1,2,3,4,5).stream() .skip(2).limit(2).toList();</pre> Output: [3, 4] Interview: Useful for in-memory paging, not database-scale pagination.
takeWhile()	dropWhile()
Explanation: Take ordered elements until first predicate failure. Input: [1,2,9,3] <pre>List<Integer> out = Stream.of(1,2,9,3) .takeWhile(n -> n < 5).toList();</pre> Output: [1, 2] Interview: Java 9; unlike filter, it stops at first false.	Explanation: Drop ordered elements until first predicate failure. Input: [1,2,9,3] <pre>List<Integer> out = Stream.of(1,2,9,3) .dropWhile(n -> n < 5).toList();</pre> Output: [9, 3] Interview: Java 9; keeps all remaining elements after failure.

3. TERMINAL OPERATIONS

forEach()

Explanation: Perform an action for each element.

Input: [A,B]

```
Stream.of("A", "B").forEach(System.out::println);
```

Output: A B

Interview: Parallel order is not guaranteed.

forEachOrdered()

Explanation: Preserve encounter order during processing.

Input: [1,2,3,4]

```
List.of(1,2,3,4).parallelStream()  
.forEachOrdered(System.out::println);
```

Output: 1 2 3 4

Interview: Ordering may reduce parallel gains.

collect()

Explanation: Gather elements into a container or grouped result.

Input: [1,2,3]

```
List<Integer> out = Stream.of(1,2,3)  
.collect(Collectors.toList());
```

Output: [1, 2, 3]

Interview: Use for List, Set, Map, grouping, and partitioning.

reduce()

Explanation: Combine all elements into one result.

Input: [10,20,30]

```
int total = Stream.of(10,20,30)  
.reduce(0, Integer::sum);
```

Output: 60

Interview: Identity must be neutral; operation associative for parallel use.

count() / min() / max()

Explanation: Count or find extreme values.

Input: [30,10,20]

```
long c = Stream.of(30,10,20).count();  
int min = Stream.of(30,10,20).min(Integer::compareTo).orElse(0);  
int max = Stream.of(30,10,20).max(Integer::compareTo).orElse(0);
```

Output: 3 / 10 / 30

Interview: min and max return Optional.

findFirst() / findAny()

Explanation: Return first ordered element or any available element.

Input: [10,20,30]

```
int first = Stream.of(10,20,30).findFirst().orElse(0);  
int any = Stream.of(10,20,30).parallel().findAny().orElse(0);
```

Output: 10 / any of 10,20,30

Interview: findAny is more parallel-friendly when order is irrelevant.

Match operations	toArray()
Explanation: Test any, all, or no elements. Input: [2,4,6] <pre>boolean any = Stream.of(2,4,6).anyMatch(n -> n > 5); boolean all = Stream.of(2,4,6).allMatch(n -> n % 2 == 0); boolean none = Stream.of(2,4,6).noneMatch(n -> n < 0);</pre> Output: true / true / true Interview: All are short-circuiting terminal operations.	Explanation: Convert a Stream into an array. Input: [A,B] <pre>String[] out = Stream.of("A","B") .toArray(String[]::new);</pre> Output: [A, B] Interview: Use the generator overload for a typed array.

4. PRIMITIVE STREAMS & COLLECTORS

mapToInt()

Explanation: Convert object Stream to IntStream.

Input: [Java, Spring]

```
int[] lengths = Stream.of("Java", "Spring")
    .mapToInt(String::length).toArray();
```

Output: [4, 6]

Interview: Avoid boxing for numeric work.

boxed()

Explanation: Convert primitive Stream to wrapper Stream.

Input: 1..3

```
List<Integer> out = IntStream.rangeClosed(1, 3)
    .boxed().toList();
```

Output: [1, 2, 3]

Interview: Required by object-based collectors.

summaryStatistics()

Explanation: Compute count, sum, min, average, max in one pass.

Input: [10, 20, 30]

```
IntSummaryStatistics s = IntStream.of(10, 20, 30)
    .summaryStatistics();
```

Output: count=3, sum=60, min=10, avg=20, max=30

Interview: Strong answer for numeric reporting.

toMap()

Explanation: Create a Map using key/value mappers.

Input: Java, Spring

```
Map<String, Integer> out = Stream.of("Java", "Spring")
    .collect(Collectors.toMap(x -> x, String::length));
```

Output: {Java=4, Spring=6}

Interview: Provide a merge function when duplicate keys are possible.

joining()

Explanation: Concatenate strings with delimiter/prefix/suffix.

Input: Java, Spring, Kafka

```
String out = Stream.of("Java", "Spring", "Kafka")
    .collect(Collectors.joining(", ", "[", "]"));
```

Output: [Java, Spring, Kafka]

Interview: Cleaner than manual StringBuilder loops.

groupingBy()

Explanation: Group values by a classifier.

Input: one, two, java

```
Map<Integer, List<String>> out = Stream.of("one", "two", "java")
    .collect(Collectors.groupingBy(String::length));
```

Output: {3=[one, two], 4=[java]}

Interview: Equivalent to SQL GROUP BY.

partitioningBy()	counting()
Explanation: Split values into true/false groups. Input: [1,2,3,4] <pre>Map<Boolean, List<Integer>> out = Stream.of(1,2,3,4) .collect(Collectors.partitioningBy(n -> n % 2 == 0));</pre> Output: {false=[1,3], true=[2,4]} Interview: Exactly two Boolean partitions.	Explanation: Count values inside a Collector. Input: a,b,a <pre>Map<String, Long> out = Stream.of("a", "b", "a") .collect(Collectors.groupingBy(x -> x, Collectors.counting()));</pre> Output: {a=2, b=1} Interview: Foundation of frequency problems.

summing / averaging	collectingAndThen()
Explanation: Aggregate mapped numeric properties. Input: [10,20,30] <pre>int sum = Stream.of(10,20,30).collect(Collectors.summingInt(Integer::intValue)); double avg = Stream.of(10,20,30).collect(Collectors.averagingInt(Integer::intValue));</pre> Output: 60 / 20.0 Interview: Use downstream inside groupingBy.	Explanation: Apply a final transformation after collection. Input: [3,1,2] <pre>List<Integer> out = Stream.of(3,1,2).collect(Collectors.collectingAndThen(Collectors.toList(), List::copyOf));</pre> Output: [3, 1, 2] unmodifiable Interview: Useful for immutable result policies.

teeing()
Explanation: Run two collectors then merge results. Input: [10,20,30] <pre>double avg = Stream.of(10,20,30).collect(Collectors.teeing(Collectors.summingInt(Integer::intValue), Collectors.counting(), (sum, count) -> sum/(double)count));</pre> Output: 20.0 Interview: Java 12; concise multi-aggregation.

5. OPTIONAL, PARALLELISM & INTERNALS

Optional.stream()

Explanation: Turn present Optional into one element, empty Optional into zero.

Input: [Optional[A], empty, Optional[B]]

```
List<String> out = Stream.of(Optional.of("A"),Optional.empty(),Optional.of("B"))
    .flatMap(Optional::stream).toList();
```

Output: [A, B]

Interview: Elegant way to remove empty Optionals.

Optional map / flatMap

Explanation: Transform value; flatMap avoids nested Optional.

Input: Optional[Java]

```
int length = Optional.of("Java")
    .map(String::length).orElse(0);
```

Output: 4

Interview: Use flatMap when mapper already returns Optional.

orElse vs orElseGet

Explanation: Eager default vs lazy Supplier default.

Input: Optional[Java]

```
String a = Optional.of("Java").orElse(expensive());
String b = Optional.of("Java").orElseGet(() -> expensive());
```

Output: Java / Java

Interview: orElse still evaluates its argument; orElseGet is lazy.

Parallel safe collection

Explanation: Build results through Stream collectors, not shared mutation.

Input: [1,2,3]

```
List<Integer> out = List.of(1,2,3).parallelStream()
    .map(n -> n * 10).toList();
```

Output: [10, 20, 30]

Interview: Avoid external ArrayList.add in parallel forEach.

Result combining

Explanation: Parallel partial results combine into final result.

Input: [1,2,3,4,5,6]

```
int sum = IntStream.rangeClosed(1,6).parallel().sum();
```

Output: 21

Interview: Combiner/reduction must be associative.

Spliterator traversal

Explanation: Process one element, then all remaining elements.

Input: [10,20,30]

```
Spliterator<Integer> sp = List.of(10,20,30).spliterator();
sp.tryAdvance(System.out::println);
sp.forEachRemaining(System.out::println);
```

Output: 10 20 30

Interview: trySplit enables parallel decomposition.

Stream fusion	Short circuiting
Explanation: Each element flows through operations without intermediate Lists. Input: [5,15,25] <pre>List<Integer> out = List.of(5,15,25).stream() .filter(n -> n > 10).map(n -> n * 2).toList();</pre> Output: [30, 50] Interview: One traversal; no separate filtered List.	Explanation: Stop once the answer is known. Input: [5,15,25] <pre>int first = List.of(5,15,25).stream() .filter(n -> n > 10).findFirst().orElse(0);</pre> Output: 15 Interview: findFirst, findAny, match operations, limit.
Immutable processing	Internal flow
Explanation: Create a new result and preserve original data. Input: [java,spring] <pre>List<String> src = List.of("java", "spring"); List<String> out = src.stream().map(String::toUpperCase).toList();</pre> Output: src=[java,spring], out=[JAVA,SPRING] Interview: Stream.toList() is unmodifiable.	Explanation: Source is traversed; lazy stages execute when terminal pulls values. Input: [1,2,3,4] <pre>List<Integer> out = List.of(1,2,3,4).stream() .filter(n -> n%2==0).map(n -> n*10).toList();</pre> Output: [20, 40] Interview: Source -> Spliterator -> stages/sinks -> terminal -> result.

6. SENIOR CODING PATTERNS

Second highest

Explanation: Distinct ranking using reverse order and skip.

Input: [10,20,40,30]

```
int out = Stream.of(10,20,40,30).distinct()
    .sorted(Comparator.reverseOrder()).skip(1).findFirst().orElse(0);
```

Output: 30

Interview: Generalize with skip(n-1) for Nth highest.

Find duplicates

Explanation: Frequency-map solution avoids shared mutable Set.

Input: [1,2,2,3,3,4]

```
List<Integer> nums = List.of(1,2,2,3,3,4);
Map<Integer,Long> f = nums.stream().collect(Collectors.groupingBy(x ->
    x,Collectors.counting()));
List<Integer> out = f.entrySet().stream().filter(e ->
    e.getValue(>1)).map(Map.Entry::getKey).toList();
```

Output: [2, 3]

Interview: More declarative and parallel-safe than external seen Set.

First non-repeated

Explanation: Preserve encounter order while counting.

Input: [1,2,2,3,3,4]

```
Integer out = nums.stream().collect(Collectors.groupingBy(x ->
    x,LinkedHashMap::new,Collectors.counting()))
    .entrySet().stream().filter(e -> e.getValue()==1)
    .map(Map.Entry::getKey).findFirst().orElse(null);
```

Output: 1

Interview: LinkedHashMap preserves first occurrence order.

Palindrome strings

Explanation: Compare each string with its reverse.

Input: level,java,madam

```
List<String> out = Stream.of("level","java","madam")
    .filter(s -> s.equals(new StringBuilder(s).reverse().toString())).toList();
```

Output: [level, madam]

Interview: Simple Predicate-based coding round pattern.

Character frequency

Explanation: Convert chars to objects, then group and count.

Input: Java

```
Map<Character,Long> out = "Java".chars().mapToObj(c -> (char)c)
    .collect(Collectors.groupingBy(c -> c,Collectors.counting()));
```

Output: {J=1, a=2, v=1}

Interview: Use LinkedHashMap supplier if output order matters.

Flatten nested list

Explanation: Use flatMap for nested collections.

Input: [[1,2],[3,4]]

```
List<Integer> out = Stream.of(List.of(1,2),List.of(3,4))
    .flatMap(List::stream).toList();
```

Output: [1, 2, 3, 4]

Interview: Classic map vs flatMap question.

Employee grouping	Highest salary by department
Explanation: Group Employees by department. Input: John-IT, David-IT, Mary-HR <pre>Map<String,List<Employee>> out = employees.stream() .collect(Collectors.groupingBy(Employee::getDepartment));</pre> Output: {IT=[John,David], HR=[Mary]} Interview: Most common enterprise Collector scenario.	Explanation: Group then use downstream maxBy. Input: John-IT-100, David-IT-150, Mary-HR-120 <pre>Map<String,Optional<Employee>> out = employees.stream().collect(Collectors.groupingBy(Employee::getDepartment, Collectors.maxBy(Comparator.comparing(Employee::getSalary))));</pre> Output: {IT=David(150), HR=Mary(120)} Interview: Follow-up: remove Optional with collectingAndThen.

List to Map	Longest string
Explanation: Map employee ID to employee name. Input: 101-John, 102-David <pre>Map<Integer,String> out = employees.stream().collect(Collectors.toMap(Employee::getId,Employee::getName));</pre> Output: {101=John, 102=David} Interview: Add merge function for duplicate IDs.	Explanation: Use comparator on String length. Input: Java,SpringBoot,Kafka <pre>String out = Stream.of("Java","SpringBoot","Kafka") .max(Comparator.comparingInt(String::length)).orElse("");</pre> Output: SpringBoot Interview: min gives shortest string.

7. HIGH-VALUE SENIOR ADDITIONS

Stream.builder()

Explanation: Build a Stream incrementally when values arrive dynamically.

Input: Java, Spring

```
List<String> out = Stream.<String>builder()
    .add("Java").add("Spring").build().toList();
```

Output: [Java, Spring]

Interview: Prefer Stream.of for fixed values; builder for incremental construction.

BufferedReader.lines()

Explanation: Turn an existing BufferedReader into a lazy line Stream.

Input: John Jane

```
try (BufferedReader br = Files.newBufferedReader(path)) {
    List<String> out = br.lines().toList();
}
```

Output: [John, Jane]

Interview: Close the reader with try-with-resources.

Collectors.mapping()

Explanation: Transform values inside a downstream grouping collector.

Input: John-IT, David-IT, Mary-HR

```
Map<String,List<String>> out = employees.stream().collect(
    Collectors.groupingBy(Employee::getDepartment,
        Collectors.mapping(Employee::getName,Collectors.toList())));
```

Output: {IT=[John,David], HR=[Mary]}

Interview: Useful when groups need selected fields rather than full objects.

Collectors.reducing()

Explanation: Perform reduction as a downstream collector inside groupingBy.

Input: IT-50, IT-70, HR-40

```
Map<String,Integer> out = employees.stream().collect(
    Collectors.groupingBy(Employee::getDepartment,
        Collectors.reducing(0,Employee::getSalary,Integer::sum)));
```

Output: {IT=120, HR=40}

Interview: reduce is standalone; reducing is a Collector.

Multi-level grouping

Explanation: Group by department and then by gender.

Input: John-IT-M, Mary-IT-F, Bob-HR-M

```
Map<String,Map<String,List<Employee>>> out = employees.stream().collect(
    Collectors.groupingBy(Employee::getDepartment,
        Collectors.groupingBy(Employee::getGender)));
```

Output: {IT={M=[John],F=[Mary]}, HR={M=[Bob]}}

Interview: Common senior-level nested Collector question.

Top N records

Explanation: Sort by business ranking and keep the first N.

Input: John-100, David-150, Mary-120

```
List<Employee> top2 = employees.stream()
    .sorted(Comparator.comparing(Employee::getSalary).reversed())
    .limit(2).toList();
```

Output: [David-150, Mary-120]

Interview: Define tie-breaking with thenComparing for deterministic results.

Java 9 iterate(predicate) Explanation: Create a finite generated sequence without a separate limit. Input: seed=1, while <=5 <pre>List<Integer> out = Stream.iterate(1,n -> n <= 5,n -> n + 1) .toList();</pre> Output: [1, 2, 3, 4, 5] Interview: Predicate controls termination before emitting the next value.	Stream.toList() immutability Explanation: Java 16 toList returns an unmodifiable List. Input: [1,2,3] <pre>List<Integer> out = Stream.of(1,2,3).toList(); out.add(4);</pre> Output: UnsupportedOperationException Interview: Do not assume Collectors.toList and Stream.toList have identical mutability.
Anagram check Explanation: Compare sorted Unicode code-point arrays. Input: listen / silent <pre>boolean out = Arrays.equals("listen".chars().sorted().toArray(), "silent".chars().sorted().toArray());</pre> Output: true Interview: Normalize case and whitespace if required by the business rule.	Word frequency Explanation: Split text, normalize words, then group and count. Input: Java Java Spring Kafka Spring <pre>Map<String,Long> out = Arrays.stream(text.split("\s+")) .collect(Collectors.groupingBy(String::toLowerCase,Collectors.counting()));</pre> Output: {java=2, spring=2, kafka=1} Interview: Define punctuation and locale normalization for production text.
Merge Lists / Streams Explanation: Use flatMap for many Lists or concat for exactly two Streams. Input: [1,2] + [3,4] <pre>List<Integer> a = Stream.of(list1,list2).flatMap(List::stream).toList(); List<Integer> b = Stream.concat(list1.stream(),list2.stream()).toList();</pre> Output: [1, 2, 3, 4] Interview: Streams are single-use; do not reuse them after concat terminal execution.	Employee salary summary Explanation: Calculate count, sum, min, average, and max in one pass. Input: 100000,150000,120000 <pre>IntSummaryStatistics stats = employees.stream() .mapToInt(Employee::getSalary).summaryStatistics();</pre> Output: count=3, sum=370000, min=100000, avg=123333.33, max=150000 Interview: Prefer primitive summary statistics for numeric reporting.

8. 60-SECOND FINAL REVISION

Term	Recall	Term	Recall
filter	Choose	findFirst	First
map	Change	findAny	Any
flatMap	Flatten	groupingBy	Many groups
distinct	Unique	partitioningBy	True / False
sorted	Arrange	boxed	Primitive to wrapper
peek	Observe	ofNullable	Null-safe Stream
limit	Take	Spliterator	Split + traverse
skip	Ignore	Fusion	One pass
collect	Gather	Short circuit	Stop early
reduce	Combine	parallelStream	Benchmark first

Final senior answer: Streams provide concise, lazy, functional pipelines. I use them for filtering, transformation, grouping, and aggregation; I avoid shared mutation, prefer primitive Streams for numeric work, close I/O Streams, and use parallel Streams only after realistic benchmarking.